COMPREHENSIVE EXERCISE

TaskFlow API

A Production-Grade Task Management REST API with Full Test Coverage

---

Courses Covered:

NodeJS – The Complete Guide (MVC, REST APIs, GraphQL, Deno)
Unit Testing for TypeScript & NodeJS Developers with Jest

Grootan Technologies – Internal Training Program

Exercise Overview

You will build TaskFlow API – a complete task management REST API using Node.js, Express, TypeScript, MongoDB/Mongoose, and JWT authentication. The project is structured into 8 progressive phases, each targeting specific course topics. Every phase requires comprehensive unit and integration tests written with Jest before or alongside the implementation code (TDD encouraged).

> 🎯 **Goal**
> Build a fully tested, production-quality REST API that covers 90%+ of concepts from both courses. Each phase has acceptance criteria – your tests must pass before moving to the next phase.

Tech Stack

Category	Technology
Runtime	Node.js 20+ with TypeScript 5+
Framework	Express.js with express-validator
Database	MongoDB with Mongoose ODM
Authentication	JWT (jsonwebtoken) + bcryptjs
Testing	Jest + ts-jest + supertest + mongodb-memory-server
Validation	express-validator + custom middleware
File Handling	multer + PDFKit (for reports)
Email	nodemailer (with test stubs)
Real-time	socket.io (for notifications)

Project Structure

Organize your project following the MVC + service layer pattern. You should have separate directories for: config, controllers (thin request handlers), middleware (auth, errors, validation, file upload), models (Mongoose schemas), routes (Express route definitions), services (business logic – the main testable units), utils (helpers, PDF generator, email sender), types (TypeScript interfaces), and socket (Socket.io event handlers). Keep your Express app creation separate from server startup so the app can be imported independently for testing.

Course Topic Mapping

Each phase maps directly to specific sections of both courses:

Phase	Node.js Course Topics	Testing Course Topics	Deliverable
1	Express.js, MVC Pattern, TypeScript	Jest setup, describe/test, assertions, ts-jest	Project scaffold + first tests
2	MongoDB, Mongoose, schemas, relations	Mocking, stubs, testing async code	Data models with full CRUD tests
3	REST APIs, routing, request parsing	Integration tests, supertest, AAA pattern	Complete CRUD endpoints + API tests
4	Authentication, JWT, bcrypt, sessions	Spying, beforeEach hooks, auth testing	Auth system with protected routes
5	Validation, error handling, status codes	Testing error paths, edge cases	Robust validation + error middleware
6	File upload/download, PDF generation	Mocking file systems, stream testing	File attachments + PDF export
7	Pagination, async/await, WebSockets	Testing pagination, socket events	Advanced queries + real-time
8	Email, deployment prep, production	E2E-style tests, coverage reports	Email + production hardening + 80% coverage

Phase 1: Project Foundation & Express Setup

Course Topics: Node.js Basics, Express.js, MVC Pattern, TypeScript Setup, Jest Configuration

> 📋 **Objective**
> Set up the TypeScript project with Express, configure Jest with ts-jest and mongodb-memory-server, implement health-check routes, and establish the MVC folder structure. Write your first unit and integration tests.

1.1 – Project Setup

Initialize a TypeScript Node.js project with Express. Install all production and dev dependencies listed in the Tech Stack table. Configure TypeScript with strict mode enabled. Separate your Express app creation from the server startup (app.ts vs server.ts) so the app can be imported for testing without starting a listener.

1.2 – Jest Configuration

- Use ts-jest as the transform preset with testEnvironment set to 'node'
- Configure path aliases so @/ maps to src/
- Set up a global test setup file that starts MongoMemoryServer before all tests and tears it down after
- Enable coverage collection with 80% threshold for branches, functions, lines, and statements

1.3 – Initial Routes & Middleware

Implement the following:

Health endpoint (GET /api/health): Return the server status, current timestamp, and uptime.

404 catch-all: Any unmatched route should return a 404 with an error message.

Global error middleware: Catch errors from next(error) and return a consistent JSON error response with the appropriate status code.

Security: Apply CORS and helmet middleware.

1.4 – Required Tests

Unit Tests:

- Test that the error handler middleware formats errors correctly for different error types
- Test that unknown errors return 500 while custom errors preserve their status codes
- Test a utility function with multiple input scenarios

Integration Tests:

- GET /api/health returns 200 with correct response shape (status, timestamp, uptime)
- GET /nonexistent returns 404 with error message
- Verify CORS headers are present
- Verify security headers (helmet) are set

✅ Acceptance Criteria

All tests pass. Jest works with TypeScript. App exports separately from server. At least 6 passing tests.

Phase 2: Data Models & Database Layer

Course Topics: MongoDB, Mongoose, Schemas, Relations, NoSQL Concepts, Mocking, Stubs, Async Test Patterns

> 📋 **Objective**
>
> Design and implement Mongoose models for Users, Tasks, Projects, and Comments with proper relations and validations. Write comprehensive unit tests for all model validations and methods.

2.1 – User Model

Design a User model with the following fields: email (unique, required, valid format), password (required, min 8 chars, stored hashed), name (required, 2–50 chars), role (enum: user/admin – default user), optional avatar path, optional resetToken and resetTokenExp for password recovery, and timestamps.

Implement instance methods: comparePassword(candidate) that returns a boolean, and generateResetToken() that creates a crypto token and sets its expiry. Add a pre-save hook that hashes the password whenever it is modified.

2.2 – Task Model

Design a Task model with: title (required, 3–100 chars), description (required, max 2000 chars), status (enum: todo/in-progress/review/done), priority (enum: low/medium/high/urgent), assignee (User ref, required), project (Project ref, required), tags (string array, max 10 tags of max 30 chars each), optional dueDate (must be future on creation), attachments array (objects with filename, path, size), optional completedAt (auto-set when status becomes done), and timestamps.

Implement static methods: findByProject(projectId) with pagination, findOverdue() returning tasks past their dueDate that are not done, and getStatusCounts(projectId) returning an object with counts per status.

2.3 – Project & Comment Models

Design a Project model with: name, description, owner (User ref), members (array of User refs), and status. Design a Comment model with: content, author (User ref), task (Task ref), and an optional parent (Comment ref) for threaded replies. Both need proper TypeScript interfaces.

2.4 – Required Tests

Test each model in isolation:

- User: validation passes/fails for email format, password length, name constraints
- User: password is hashed on save, never stored in plain text

- User: comparePassword returns true for correct password, false for incorrect
- User: generateResetToken creates a token and sets an expiry
- Task: validation enforces required fields, enum values, and tag limits
- Task: dueDate must be a future date – past dates are rejected
- Task: completedAt auto-populates when status changes to 'done'
- Task: static method findOverdue returns correct results
- Task: static method getStatusCounts aggregates correctly
- Project/Comment: relation refs are properly validated

✅ Acceptance Criteria

All 4 models implemented with TypeScript interfaces. At least 15 model tests passing. Password hashing verified. All static methods tested with mongodb-memory-server.

Phase 3: REST API Endpoints & CRUD

Course Topics: REST API Design, Express Routing, Request Parsing, Controllers, Services Pattern, Integration Testing with Supertest

> 📋 **Objective**
>
> Build complete CRUD REST endpoints for Tasks, Projects, and Comments following the service layer pattern. Controllers should be thin – they parse the request and delegate to a service. The service contains business logic and is the main target for unit tests. Write integration tests for every endpoint.

3.1 – API Endpoints

1. **POST /api/tasks** – Create a new task
2. **GET /api/tasks** – List tasks with query filters (status, priority, assignee) and pagination
3. **GET /api/tasks/:id** – Get single task with populated references (assignee, project, comments)
4. **PUT /api/tasks/:id** – Update task (partial update supported)
5. **DELETE /api/tasks/:id** – Soft-delete task (set a deletedAt timestamp)
6. **POST /api/projects** – Create a new project
7. **GET /api/projects/:id/tasks** – List all tasks belonging to a project
8. **POST /api/tasks/:id/comments** – Add a comment to a task (support threaded replies via parent ref)
9. **GET /api/tasks/:id/comments** – Get threaded comments for a task

3.2 – Required Tests

Unit Tests (mock the database layer):

- TaskService.create – verify correct data flows to the database layer
- TaskService.create – throws 404 when project is not found
- TaskService.create – throws 403 when user is not a project member
- TaskService.findAll – pagination parameters are passed correctly
- TaskService.update – partial updates merge correctly with existing data
- TaskService.delete – soft-delete sets deletedAt instead of removing the document

Integration Tests (supertest + memory DB):

- POST /api/tasks with valid data returns 201 and the created task
- POST /api/tasks with missing required fields returns 400 with error details
- GET /api/tasks returns paginated list with correct pagination metadata
- GET /api/tasks?status=todo filters results correctly
- GET /api/tasks/:id with valid ID returns task with populated references

- GET /api/tasks/:id with non-existent ID returns 404
- PUT /api/tasks/:id updates only the provided fields
- DELETE /api/tasks/:id returns 200 and task is soft-deleted (still in DB but marked)
- POST /api/tasks/:id/comments creates a comment linked to the task
- GET /api/projects/:id/tasks returns only tasks belonging to that project

✅ Acceptance Criteria

All endpoints functional. At least 20 tests passing (6+ unit, 10+ integration). Service layer fully separated from controllers. Pagination works with page/limit query params.

Phase 4: Authentication & Authorization

Course Topics: JWT, bcrypt, Auth Middleware, Route Protection, Advanced Authentication, Password Reset, Spying, beforeEach Hooks

> 📋 **Objective**
>
> Implement a complete authentication system with JWT access tokens, refresh tokens, and password reset flow. Add authorization middleware to protect routes. Update all Phase 3 tests to include auth headers.

4.1 – Auth Endpoints

- **POST /api/auth/signup** – Register new user, hash password, return JWT
- **POST /api/auth/login** – Verify credentials, return JWT + refresh token
- **POST /api/auth/refresh** – Issue new access token using a valid refresh token
- **POST /api/auth/forgot-password** – Generate reset token, send email
- **POST /api/auth/reset-password/:token** – Reset password using valid token
- **GET /api/auth/me** – Get current user profile (protected route)

4.2 – Middleware & Authorization

- **isAuth middleware:** Extract JWT from Authorization: Bearer header, verify, attach userId to request
- **isAdmin middleware:** Check user role is 'admin' after isAuth has run
- **isOwnerOrAdmin:** For task operations, verify user is the assignee or an admin
- **Rate limiting:** Apply to auth routes (e.g., 5 attempts per 15 minutes)

4.3 – Required Tests

Unit Tests:

- isAuth: passes when a valid JWT is provided
- isAuth: returns 401 for missing token, expired token, and invalid token
- isAdmin: passes for admin role, returns 403 for regular user role
- AuthService.signup: hashes the password and returns a JWT
- AuthService.login: throws 401 for wrong email and wrong password separately
- AuthService.resetPassword: throws 400 for expired or invalid tokens
- AuthService.forgotPassword: triggers the email service with the correct reset link

Integration Tests:

- Full signup flow: register → receive JWT → use JWT to access /api/auth/me successfully
- Login with correct credentials returns a JWT
- Login with wrong password returns 401
- Accessing a protected route without a token returns 401
- Accessing an admin-only route as a regular user returns 403
- Password reset flow: forgot-password → use token → reset-password → login with new password
- Refresh token generates a new valid access token

✅ Acceptance Criteria

Complete auth system with JWT + refresh tokens. All Phase 3 integration tests updated to include auth. Password reset works end-to-end. At least 14 new auth tests passing.

Phase 5: Validation & Error Handling

Course Topics: express-validator, Custom Validators, Async Validation, Sanitization, Error Types, Express Error Middleware, Status Codes

> 📋 **Objective**
> Add thorough input validation to every endpoint, implement a custom error class and centralized error handling middleware that gracefully handles all error types.

5.1 – Validation Rules

- Implement express-validator chains for every POST and PUT endpoint
- Title: required, trimmed, 3–100 characters
- Description: required, max 2000 characters
- Priority: must be one of the allowed enum values
- Project ID: must be a valid MongoDB ObjectId and must reference an existing project (async validator)
- Due date: optional, but if provided must be a valid ISO date in the future
- Tags: optional array, max 10 items, each tag max 30 characters
- Email fields: must be valid email format, normalized to lowercase
- Sanitize all string inputs to strip HTML and script tags

5.2 – Custom Error System

- Create a custom AppError class with message, statusCode, and optional validation errors array
- Global error handler middleware that catches all errors and returns a consistent JSON shape
- Handle Mongoose ValidationError → return 400 with field-level error messages
- Handle Mongoose CastError (invalid ObjectId) → return 400
- Handle MongoDB duplicate key error (code 11000) → return 409 with the conflicting field name
- Handle JWT TokenExpiredError → return 401
- Handle JWT JsonWebTokenError → return 401
- Hide stack traces in production, include them in development

5.3 – Required Tests

- Unit test each custom validator function (future date check, tag limit, etc.)
- Unit test AppError class instantiation and that it carries the correct properties
- Unit test error handler: Mongoose ValidationError maps to 400 with field errors

- Unit test error handler: CastError maps to 400
- Unit test error handler: duplicate key error maps to 409
- Unit test error handler: JWT errors map to 401
- Integration test: POST endpoints with each type of invalid field return the specific error message
- Integration test: verify sanitization strips HTML/script tags from inputs
- Integration test: verify correct status codes are returned (400, 401, 403, 404, 409, 500)

✅ Acceptance Criteria

Every endpoint has validation. Custom AppError used throughout. Error handler covers Mongoose, JWT, and custom errors. At least 12 new tests. No unhandled promise rejections.

Phase 6: File Upload, Download & PDF Generation

Course Topics: Multer, File Upload/Download, Static Files, PDF Generation (PDFKit), Streams, Mocking File System

📋 **Objective**

Implement file upload/download for task attachments and user avatars, generate project report PDFs on-the-fly, and export tasks as CSV.

6.1 – Features

1. **Task Attachments (POST /api/tasks/:id/attachments):** Upload files using multer. Max 5MB, allowed types: pdf, png, jpg, docx. Store metadata (filename, path, size) in the task's attachments array.
2. **Download Attachment (GET /api/tasks/:id/attachments/:attachmentId):** Stream the file back to the client with proper Content-Type and Content-Disposition headers.
3. **User Avatar (PUT /api/auth/me/avatar):** Upload and optionally resize a profile image.
4. **Project Report PDF (GET /api/projects/:id/report):** Generate a PDF on-the-fly containing: project name, member list, task counts by status, and a list of overdue tasks. Stream it as the response.
5. **Task Export CSV (GET /api/projects/:id/export?format=csv):** Export all project tasks as a downloadable CSV file.

6.2 – Required Tests

- Unit test: file upload config rejects files over 5MB
- Unit test: file upload config rejects disallowed file types (e.g., .exe)
- Unit test: PDF generation service produces a valid buffer with expected content
- Unit test: CSV generation produces correct header row and data formatting
- Integration test: upload a file to a task, then download it and verify the content matches
- Integration test: uploading an oversized file returns 400
- Integration test: GET /api/projects/:id/report returns application/pdf content-type
- Integration test: upload avatar, then GET /api/auth/me returns the avatar URL

✅ **Acceptance Criteria**

File upload/download works with proper validation. PDF report generates dynamically. CSV export functional. At least 8 new tests.

Phase 7: Advanced Pagination & Real-time

Course Topics: Pagination, Async/Await, WebSockets (Socket.io), Testing Async Operations, Testing External Integrations

> 📋 **Objective**
>
> Implement both offset-based and cursor-based pagination with sorting. Add real-time notifications using Socket.io so project members are instantly notified of task changes.

7.1 – Advanced Pagination & Sorting

- Implement both offset-based (page/limit) and cursor-based (cursor/limit) pagination on task listing endpoints
- Offset response includes: total, page, limit, totalPages, hasMore
- Cursor response includes: nextCursor, hasMore, limit
- Handle edge cases: page beyond total, empty results, limit of 0 or negative values
- Add sorting support: sort by createdAt, dueDate, priority (ascending/descending via query param)
- Add combined filtering + pagination + sorting in a single query

7.2 – Real-time Notifications (Socket.io)

- **Socket.io integration:** When a task is created, updated, assigned, or commented on, emit an event to the relevant project room
- **Events to implement:** task:created, task:updated, task:assigned, task:commented, task:status-changed
- **Rooms:** Users auto-join rooms by their project IDs. Only project members receive events.
- **Auth:** Socket connections must include a JWT in the handshake query or auth object. Reject unauthenticated connections.

7.3 – Required Tests

- Unit test: cursor-based pagination returns correct nextCursor and hasMore values
- Unit test: offset pagination calculates totalPages correctly (0 items, exact multiples, partial pages)
- Unit test: socket event emitter is called with correct event name and payload when a task is created
- Unit test: socket auth rejects connections without a valid JWT
- Integration test: pagination with various page/limit combinations returns correct slices
- Integration test: cursor pagination can traverse the full dataset without duplicates or gaps
- Integration test: sorting combined with filtering returns results in the correct order

- Integration test: socket client receives task:created event after POST /api/tasks
- Integration test: unauthenticated socket connection is rejected

✅ Acceptance Criteria

Both pagination modes work correctly. Socket.io events fire for task mutations. Auth enforced on socket connections. At least 10 new tests.

Phase 8: Email Notifications & Production Readiness

Course Topics: Nodemailer, Sending Emails, Deployment Preparation, Compression, Logging, Coverage Reports

> 📋 **Objective**
>
> Add email notifications using nodemailer, apply production-hardening middleware, and ensure the full test suite achieves 80%+ coverage.

8.1 – Email Notifications

Using nodemailer (with a mock/test transport in the test environment so no real emails are sent), implement emails for:

1. Welcome email on signup (with a verification link)
2. Password reset email (with the token link)
3. Task assignment notification ("You have been assigned to: [task name]")
4. Daily digest: a summary of overdue tasks for each user (design how this would be triggered – cron job or manual endpoint)

8.2 – Production Hardening

- Add request logging (morgan or a custom middleware)
- Add response compression
- Add global and per-route rate limiting
- Add a request ID middleware for tracing (attach a UUID to each request and include it in logs and responses)
- Environment-specific configuration (development, test, production) – no hardcoded values
- Graceful shutdown handling: close DB connections, drain socket.io, finish in-flight requests

8.3 – Required Tests

- Unit test: email service composes correct HTML for each email type (verify subject, recipient, body content)
- Unit test: email service uses a mock transport in test environment and never sends real emails
- Unit test: daily digest logic correctly identifies users with overdue tasks
- Integration test: full user journey – signup → create project → create task → add comment → upload file → verify everything works end-to-end
- Integration test: rate limiting blocks requests after the threshold is exceeded
- Verify total test coverage is above 80% for lines, branches, functions, and statements

✅ Acceptance Criteria

Emails are sent via mocked transport in tests. Production middleware active. Full user journey test passes. Overall test coverage >80%. Total test count across all phases: 80+.

Bonus Challenges (Optional)

For those who want to push further after completing all 8 phases:

Challenge A: GraphQL Layer

Add a GraphQL API alongside the REST API (both should work simultaneously):

- Implement Query types for tasks, projects, and users with field-level resolvers
- Implement Mutations for CRUD operations with input validation
- Add authentication to GraphQL context
- Write tests for resolvers

Challenge B: Background Job Queue

Add Bull or BullMQ for background processing:

- Move email sending to a background job queue
- Add a scheduled daily digest job
- Add PDF report generation as an async job (return a job ID, let the client poll for completion)
- Test job processors in isolation with a mocked queue

Challenge C: Swagger API Documentation

Add interactive API documentation using Swagger/OpenAPI so anyone can explore and test your API from the browser:

- Integrate swagger-jsdoc and swagger-ui-express to auto-generate interactive API documentation
- Add JSDoc/OpenAPI annotations to every route covering request parameters, request body schemas, response shapes, and status codes
- Document authentication requirements (Bearer token) so Swagger UI allows testing protected endpoints directly
- Include example request/response payloads for each endpoint
- Ensure the Swagger UI is accessible at /api-docs and stays in sync as you add or modify routes

Deliverables & Submission Checklist

Upon completion, you must submit the following deliverables for review. Each item will be verified before the exercise is considered complete.

1. Source Code Repository

- A single Git repository (GitHub/GitLab) with the complete project source code
- Clean commit history showing incremental progress across all 8 phases (avoid single giant commits)
- A well-written README.md that includes: project description, setup instructions, how to run the app, and how to run the tests
- A .env.example file listing all required environment variables (without actual secrets)
- No node_modules, .env, or build artifacts committed (proper .gitignore in place)

2. Working Application

- The application must start without errors using a single command (e.g., npm run dev)
- All API endpoints must be accessible and respond correctly (demonstrate with Postman, Thunder Client, or similar)
- A Postman collection or equivalent API documentation file exported and included in the repo covering all endpoints with example requests and responses
- MongoDB connection works (local or Atlas – instructions in README)
- Socket.io connection can be established and events are received by a test client

3. Test Suite

- All tests pass when running npm test with zero failures
- Test coverage report generated (npm run test:coverage) showing 80%+ on lines, branches, functions, and statements
- Coverage HTML report included or accessible via command
- Minimum 80 total tests across all phases
- Both unit tests (with mocks/stubs) and integration tests (with supertest + memory DB) are present
- No tests that are skipped, commented out, or trivially passing (e.g., expect(true).toBe(true))

4. Phase-wise Demo

Prepare a short walkthrough (live demo or screen recording of 10–15 minutes) covering:

1. User signup and login flow – show the JWT being returned and used for subsequent requests
2. Create a project, add tasks with different statuses and priorities, and demonstrate filtering and pagination
3. Demonstrate validation by sending invalid data and showing the error responses
4. Upload a file attachment to a task and download it back
5. Generate a project report PDF and show the output
6. Show a Socket.io client receiving a real-time event when a task is created or updated
7. Run the full test suite and show the coverage summary output

🛠️ Validation Checklist (Reviewer Use)

1) git clone + npm install runs cleanly 2) npm run dev starts the server 3) npm test passes with 0 failures 4) npm run test:coverage shows ≥80% across all metrics 5) Postman collection covers all endpoints 6) Demo covers all 7 walkthrough items

Evaluation Rubric

Criteria	Weight	What We Look For
Test Quality	30%	Meaningful assertions, edge cases, proper mocking/stubbing, no false positives
Code Architecture	25%	Clean MVC/service separation, TypeScript types used correctly, DRY principles
API Design	20%	RESTful conventions, consistent responses, proper status codes, pagination
Feature Completeness	15%	All 8 phases implemented and working end-to-end
Error Handling	10%	Graceful errors, no unhandled rejections, proper logging

💡 Tips for Success

Write tests first (TDD) whenever possible – it will force you to think about the API contract before implementation. Keep commits small and frequent. Finish each phase completely before moving to the next. When stuck, refer back to the relevant course section.